



ΗΥ340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ,
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ,
ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ

```
VAR i:Integer;  
    FUNCTION(Symbol) replicate  
        x = (function(x,y){return x+y;});  
        class DelFunctor: public std::unary_function<
```

ΔΙΔΑΣΚΩΝ

Αντώνιος Σαββίδης



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

Διάλεξη 1η ΕΙΣΑΓΩΓΗ



Σχετικά με το μάθημα (1/3)

- Περίπου είκοσι (24) διαλέξεις – δείτε το ημερολόγιο
- Μία βασική εργασία - project
 - Πέντε φάσεις παράδοσης (η 4η και 5η μαζί)
 - Ομάδες τριών (3) ατόμων (το πολύ)
 - Αυτόματος και αυστηρός έλεγχος αντιγραφών, αντιγραφή σημαίνει *μηδενισμός όλης της εργασίας*
 - ◆ Για τη δυναμική γλώσσα alpha – έχουμε όλα τα προηγούμενα projects και θα συγκρίνουμε αυτομάτως και με αυτά
- Σημειώσεις είναι κυρίως τα περιεχόμενα των διαλέξεων
 - θα χρησιμοποιηθούν και ορισμένα επιλεγμένα αποσπάσματα από βιβλίο (~15 σελίδες συνολικά)
 - θα βρίσκονται στο site του μαθήματος την επομένη της παράδοσης σε PowerPoint και PDF (2 διαφάνειες / σελίδα)
- Δεν υπάρχει εξέταση και ο βαθμός προκύπτει αποκλειστικά από την εργασία ως εξής:
 - **Βαθμός = Εργασία 100%**



Σχετικά με το μάθημα (2/3)

- Το μάθημα είναι ιδιαίτερα δύσκολο, καθώς περιέχει αρκετή αλλά και πολύ πυκνή «ύλη»
- Η εργασία γενικά θεωρείται η δυσκολότερη σε προπτυχιακό επίπεδο (όμως δεν είναι)
 - συνήθως είναι δύσκολο να αντιμετωπιστεί εάν αρχίσετε αργά
 - θα απαιτήσει να προγραμματίσετε αρκετά και σε ποσότητα αλλά και σε ποιότητα
 - ◆ τα όποια λάθη σας, όσο μικρής έκτασης και να είναι, θα διαπιστώσετε ότι θα σας «παιδεύουν» πολύ – μη βιάζεστε!!
- Η εργασία σας υποστηρίζεται πάρα πολύ από τις σημειώσεις και το διδακτικό υλικό
 - θα υπάρχουν μικρές και σχετικά σημαντικές αποκλίσεις στον ορισμό της εργασίας από τις διαλέξεις



Σχετικά με το μάθημα (3/3)

- Στο μάθημα αυτό επιπλέον αποκρυπτογραφούμε τους βασικούς μηχανισμούς των γλωσσών προγραμματισμού
 - εάν φέρετε σε πέρας επιτυχώς την εργασίας σας, εκ των πραγμάτων προάγεστε σε ένα υψηλότερο επίπεδο ικανοτήτων προγραμματισμού
- Οι τεχνικές έχουν πολλές χρήσεις και εφαρμογές, χωρίς να περιορίζονται απλώς στην κατασκευή γλωσσών
- Η παρακολούθηση δεν είναι υποχρεωτική, ωστόσο είναι απολύτως απαραίτητη (αν ξυπνάτε σχετικά νωρίς)
 - Οι διαλέξεις δεν έχουν αυτοτελές και ανεξάρτητο περιεχόμενο, αλλά εξαρτώνται σε μεγάλο βαθμό από προηγούμενες
- Στην αρχή να κινηθούμε περισσότερο σε θεωρία και γρήγορα σε κατασκευαστικά θέματα



Περιεχόμενα

- *Ο ρόλος του μεταγλωττιστή και οι φάσεις μετάφρασης*
- Γλώσσες προγραμματισμού
- Τύποι και μεταβλητές
- Static and dynamic typing
- Κύκλοι επεξεργασίας στη μετάφραση
- Διερμηνείς και μεταφραστές
- Παραγωγή κώδικα για πραγματικές ή εικονικές μηχανές
- Οι μεταγλωττιστές υπάρχουν παντού

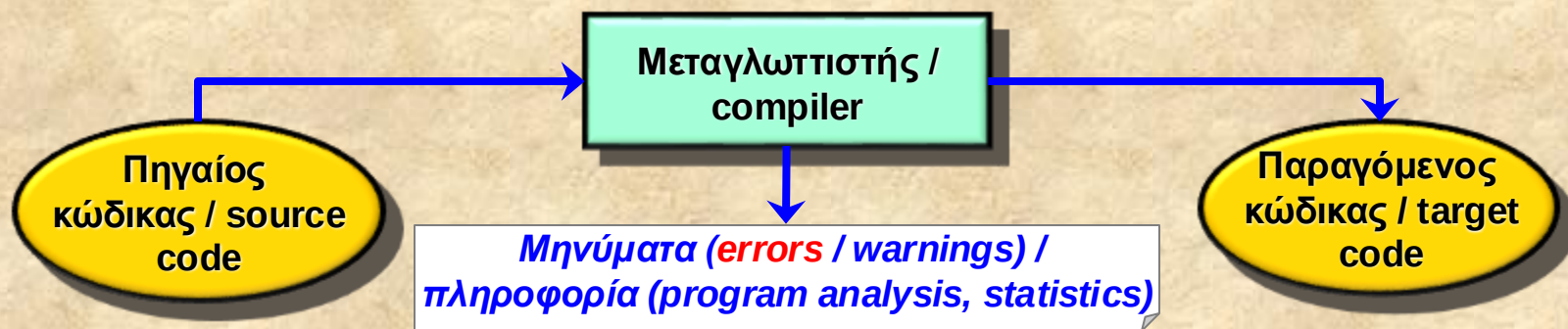


Ο ρόλος του μεταγλωττιστή (1/2)

- Ο βασικός σκοπός ύπαρξης ενός μεταγλωττιστή είναι η μετάφραση ενός προγράμματος από μία γλώσσα **πηγής** σε μία γλώσσα **προορισμού**
 - Συνήθως, χωρίς αυτό να είναι κανόνας:
 - ◆ Πηγή = γλώσσα υψηλού επιπέδου (π.χ. C, Pascal, C++, Java)
 - ◆ Προορισμός = γλώσσα κάποιας μηχανής (π.χ. Pentium, MIPS, JVM)
- Η ύπαρξη των μεταγλωττιστών υπήρξε καθοριστικός παράγοντας προόδου στην επιστήμη υπολογιστών
 - επειδή προσφέρει τη δυνατότητα προγραμματισμού σε γλώσσες υψηλού επιπέδου
 - ◆ άνοιξε το δρόμο κατασκευής μεγαλύτερων προγραμμάτων
 - ◆ σε μικρότερο χρόνο
 - ◆ με μείωση του κόστους ανάπτυξης
 - ◆ και αύξηση της «διάρκειας ζωής» των προγραμμάτων

Ο ρόλος του μεταγλωττιστή (2/2)

- Γενικά, ο λειτουργικός ρόλος του μεταγλωττιστή είναι αυτός του παρακάτω σχήματος
 - Εάν η γλώσσα προορισμού είναι και αυτή γλώσσα υψηλού επιπέδου μπορεί να χρησιμοποιείται ο όρος *meta-compiler* ή *pre-compiler*
 - ◆ Ωστόσο αυτό δεν σημαίνει κάποια ιδιαίτερη τακτική κατασκευής ούτε προμηνύει ευκολότερη ή δυσκολότερη κατασκευή





Περιεχόμενα

- Ο ρόλος του μεταγλωττιστή και οι φάσεις μετάφρασης
- *Γλώσσες προγραμματισμού*
- Τύποι και μεταβλητές
- Static and dynamic typing
- Κύκλοι επεξεργασίας στη μετάφραση
- Διερμηνείς και μεταφραστές
- Παραγωγή κώδικα για πραγματικές ή εικονικές μηχανές
- Οι μεταγλωττιστές υπάρχουν παντού



Γλώσσες προγραμματισμού (1/5)

- Ακολουθεί μία μερική κατηγοριοποίηση με σκοπό να αναδείξει τη διαφορετική φύση ορισμένων γλωσσών

Κατηγορία	Χαρακτηριστικά
Προστακτική γλώσσα / imperative language	Περιγραφή εντολών που πρέπει να εκτελεστούν επακριβώς, μνήμη → <i>εντολές, μεταβλητές, store</i>
Περιγραφή δομής / structure definition	Λεπτομερής δομική περιγραφή κάποιων αντικειμένων → <i>ιεραρχία, χαρακτηριστικά</i>
Περιγραφή προτύπων / pattern definition	Περιγραφή κάποιων επαναλαμβανόμενων μορφών → <i>τυπικές γλώσσες</i>
Λογικού προγραμματισμού / logic programming	Περιγραφή ιδιοτήτων, γεγονότων → <i>ορισμός λύσεων με λογικές εκφράσεις αναδρομικά</i>
Συναρτησιακού προγραμματισμού / functional programming	Μαθηματικός ορισμός συναρτήσεων → <i>δεν υπάρχουν μεταβλητές / εντολές, τιμές δίνονται μόνο με κλήσεις συναρτήσεων, έμφαση στην αναδρομή</i>



Γλώσσες προγραμματισμού (2/5)





Γλώσσες προγραμματισμού (3/5)

Scheme (functional)

```
(define pi 3.14159)
(define circumference (radius)(* 2 pi radius))
```

C (imperative)

```
#define pi 3.14159
#define circumference(radius) \
    (2 * pi * (radius))
double circumference (double radius) {
    return 2 * pi * radius;
}
```

Ls / grep, pattern matching, regular expressions

```
ls a???_prj?.c?
grep '[^:]*::' /etc/passwd
```

HTML, ιεραρχικές δομές με συνδέσμους

```
<table border="3" cellspacing="1" cellpadding="1">
<caption> <font size+=2> <b>Aircrafts</b> </font> </caption>
<tr> <b>Craft type</b> <b>Company</b> </tr>
<tr> F-16 Falcon <td> McDonnell Douglas </tr>
<tr> F-14 Tomcat <td> McDonnell Douglas </tr>
<tr> F-18 Hornet <td> General Dynamics </tr>
</tr> </td> </table>
```



Γλώσσες προγραμματισμού (4/5)

- Κάποια σημαντικά αποφθέγματα (Alan Perlis, ALGOL team, ο πρώτος που βραβεύτηκε με το Turing Award, 1966) από το *Epigrams on Programming*
 - Μία γλώσσα που δεν επηρεάζει τον τρόπο που αντιλαμβάνεσαι τον προγραμματισμό δεν αξίζει τον κόπο να τη μάθεις
 - Για να κατανοήσεις πλήρως ένα πρόγραμμα πρέπει να μπορείς να γίνεις τόσο η μηχανή όσο και το πρόγραμμα
 - Θα υπάρχουν πάντα πράγματα που θα επιθυμούμε να εκφράσουμε στα προγράμματά μας τα οποία σε όλες τις γνωστές γλώσσες μπορούν να ειπωθούν με φτωχό τρόπο
 - Στον προγραμματισμό ότι κάνουμε είναι ειδική περίπτωση από κάτι πιο γενικό και συνήθως το αντιλαμβανόμαστε πολύ πρόωρα
 - Δεν είναι τα δυνατά σημεία μίας γλώσσας αλλά οι αδυναμίες της που ρυθμίζουν το ρυθμό μεταβολής της
 - Ένας χρόνος ενασχόλησης με την Τεχνητή Νοημοσύνη είναι επαρκής για να κάνουν κάποιον να πιστέψει στον Θεό
 - Όταν κάποιος πει «Θέλω μία γλώσσα στην οποία απλώς να χρειάζεται να πω τι επιθυμώ να γίνει» → δώστε του μία καραμέλα



Γλώσσες προγραμματισμού (5/5)

- Επηρεάζουν σημαντικά τον τρόπο που ο προγραμματιστής σκέφτεται και σχεδιάζει ένα σύστημα
- Μπορεί να εισάγουν προγραμματιστικά στοιχεία που προάγουν σημαντικά την τεχνολογία λογισμικού
- Σήμερα ορισμένοι βασικοί στόχοι είναι η δημιουργία γλωσσών που υποστηρίζουν: ***metaprogramming, polymorphism, simplicity, safety, robustness, evolution, maintenance***
- Απίθανο κάποτε να κυριαρχήσει μόνο μία γλώσσα
 - *there cannot be only one*
- Το «Άγιο Δισκοπότηρο» των γλωσσών προγραμματισμού είναι η έννοια του verifying compiler: με είσοδο ένα πρόγραμμα P απαντάει στο ερώτημα «το P είναι σωστό;»
 - Το κύριο πρόβλημά είναι πρακτικό: εκτός του προγράμματος εισόδου μάλλον απαιτούνται τυπικοί ορισμοί με πολυπλοκότητα κατασκευής μεγαλύτερη από αυτή του προγράμματος P
 - Χωρίς ιδιαίτερη αξία πλέον...



Περιεχόμενα

- Ο ρόλος του μεταγλωττιστή και οι φάσεις μετάφρασης
- Γλώσσες προγραμματισμού
- *Τύποι και μεταβλητές*
- Static and dynamic typing
- Κύκλοι επεξεργασίας στη μετάφραση
- Διερμηνείς και μεταφραστές
- Παραγωγή κώδικα για πραγματικές ή εικονικές μηχανές
- Οι μεταγλωττιστές υπάρχουν παντού



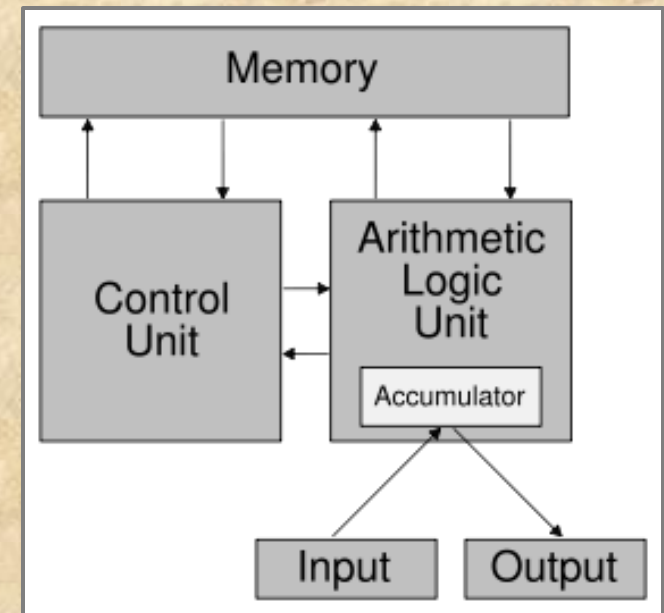
Τύποι και μεταβλητές (1/16)

- Οι μεταγλωττιστές - μεταφραστές αφορούν την ευρύτερη οικογένεια γλωσσών, για οποιονδήποτε σκοπό χρήσης
 - και όχι αποκλειστικά τις προστακτικές τις γλώσσες προγραμματισμού
- Θα επικεντρωθούμε σε ορισμένα θεμελιώδη στοιχεία των προστακτικών γλωσσών προγραμματισμού:
 - *Τύποι δεδομένων*
 - *Μεταβλητές*

Τύποι και μεταβλητές (2/16)

■ Μνήμη - store

- Στο θεμελιώδες υπολογιστικό μοντέλο *von Neumann* ένα πρόγραμμα συνίσταται σε εντολές που μετασχηματίζουν μία μνήμη – *store*
- Η μνήμη περιγράφεται ως συλλογή από διακριτά ζεύγη της μορφής **<address: value>**
- Προστακτικές (imperative) είναι οι γλώσσες που προσφέρουν εντολές οι οποίες αλλάζουν (read / write) το *store*, το οποίο ονομάζεται και *program state*





Τύποι και μεταβλητές (3/16)

■ Μεταβλητές - *variables*

- Οι μεταβλητές είναι ουσιαστικά μία μέθοδος αφαίρεσης ή συμβολικής αναφοράς στη μνήμη
- Μία μεταβλητή αντιστοιχεί σε ένα χώρο μνήμης τον οποίο χειρίζεται το πρόγραμμα
- Οι μεταβλητές έχουν διάφορα χαρακτηριστικά όπως:
 - ◆ Αναγνωριστικό όνομα - *name*
 - ◆ Συντακτική εμβέλεια - *scope*
 - ◆ Χρόνο ζωής - *lifetime*
 - ◆ Τιμή - *value*
 - ◆ Τύπο - *type*



Τύποι και μεταβλητές (4/16)

■ Ονόματα – *names*

- Συνήθως λέγονται και ταυτότητες - *identifiers*
- Χρησιμοποιούνται για να ταυτοποιούμε τις περισσότερες οντότητες των γλωσσών
 - ◆ *vars, types, funcs, classes, packages, namespaces, ...*
- Εξαρτώνται ή όχι από τη χρήση κεφαλαίων ή μικρών γραμμάτων (case sensitivity) και μπορεί να επιτρέπουν ή όχι ειδικούς χαρακτήρες (underscores, dollars, κλπ)
- Ίσως να επιβάλλουν και μέγιστο αριθμό χαρακτήρων
- Οι λέξεις κλειδιά - *keywords* – είναι ειδικά ονόματα με σημασιολογία συγκεκριμένη στη γλώσσα (πχ., *if, else*) και δεν επιτρέπεται να χρησιμοποιούνται ως αναγνωριστικά ονόματα



Τύποι και μεταβλητές (5/16)

■ Εμβέλεια – *scope*

- Σύνολο εντολών του προγράμματος, συνήθως συνεχόμενων, μέσα στις οποίες μία μεταβλητή είναι προσβάσιμη (ας πούμε «επιτρέπεται» η χρήση της)
- Μπορεί να είναι global (file), local (block or functions), ή και non-local (other blocks or functions)
- **Δυναμική εμβέλεια – dynamic scoping**
 - ◆ Ορίζεται βάσει της εκτέλεσης του προγράμματος: η εμβέλεια μίας μεταβλητής ισχύει έως ότου μία δήλωση με το ίδιο όνομα συναντάται κατά την εκτέλεση, δηλ. οι δηλώσεις είναι εντολές, με πολύ εύκολη υλοποίηση
 - 1. για κάθε αναγνωριστικό ***x*** διατηρείται ένα global stack ***x.stack***
 - 2. η εκτέλεση ενός ***decl x*** κάνει push στο ***x.stack*** (που μπορεί να ήταν άδειο) νέο χώρο μνήμης
 - 3. όταν ο έλεγχος φεύγει από scope όπου υπήρχε μία δήλωση ***decl x*** κάνουμε pop από το ***x.stack***
 - 4. κάθε αναφορά στο ***x*** είναι συνώνυμο του ***x.stack.top()***
 - ◆ Γλώσσες που υιοθετούν dynamic scoping είναι Apl, Lisp, Snobol και παλαιότερες εκδόσεις της Lua



Τύποι και μεταβλητές (6/16)

- **Στατική εμβέλεια - static scoping ή lexical scoping**
 - ◆ Η εμβέλεια ορίζεται βάσει της λεκτικής και συντακτικής δομής του προγράμματος
 - ◆ Ένα όνομα αναφέρεται πάντοτε στην πλησιέστερη δήλωση μεταβλητής (συνήθως προς τα πάνω στο κείμενο)
 - ◆ Συνήθως πολύ ταχύτερο στην εκτέλεση καθώς η αντιστοίχιση γίνεται *at compile time*
 - ◆ Η απόλυτη ή σχετική διεύθυνση μνήμης στην οποία αναφέρεται κάθε όνομα είναι γνωστή πριν την εκτέλεση κώδικα που χρησιμοποιεί τη μεταβλητή
 - ◆ Γλώσσες που υιοθετούν lexical scoping είναι: C++, Ada, C#, Java, Python, alpha, Delta, πρόσφατες εκδόσεις της Lua



Τύποι και μεταβλητές (7/16)

```
string a = "hello";  
f() { print(a); }  
g() {  
    string a = "world";  
    f();  
}  
call g();
```

• Στο διπλανό παράδειγμα, για μία γλώσσα όπως η C, ξέρουμε όλοι ότι η g() εκτυπώνει "hello"

• Όμως το ίδιο πρόγραμμα σε μία γλώσσα με dynamic scoping θα εκτυπώσει "world"

Εκτέλεση με dynamic scoping

1: string a = "hello";
 a.stack.push("hello") →



2: g();

3: string a = "world";
 a.stack.push("world") →



4: f();

5: print a;
 print a.stack.top()



Τύποι και μεταβλητές (8/16)

■ Διάρκεια ζωής μεταβλητών – *lifetime*

- Το διάστημα χρόνου κατά το οποίο η περιοχή μνήμης (όπου αποθηκεύεται η τιμή) δεσμεύεται για μία μεταβλητή
- Αυτή η περιοχή ονομάζεται αντικείμενο δεδομένων - *data object*
- Η δέσμευση χώρου μνήμης για μία μεταβλητή λέγεται παραχώρηση μνήμης - *allocation*
- Η αντιστοίχιση μίας μεταβλητής σε χώρο μνήμης λέγεται *storage binding*

■ Στατική παραχώρηση - **static allocation**: πριν την εκτέλεση

- *static / global variables in C / C++ / Java*

■ Ημιστατική παραχώρηση - **semi-static allocation**: κατά την εκτέλεση στο χώρο του *activation record*

- *Local variables in C / C++*

■ Ημιδυναμική παραχώρηση - **semi-dynamic allocation**: κατά την εκτέλεση στο χώρο πάνω από το *activation record*, συνήθως για γλώσσες με *dynamic scoping*

- *Logo, Perl (υποστηρίζει και static allocation)*

■ Δυναμική παραχώρηση - **dynamic allocation**: κατά την εκτέλεση από το σωρό (*heap*)

- *Objects in Java (new), dynamic objects in C++ (new), pointed variables in C (*p)*



Τύποι και μεταβλητές (9/16)

■ Τιμές – *values*

- Συνήθως δυαδική τιμή ή τιμή σε κάποια εσωτερική κωδικοποίηση
- Το περιεχόμενο ερμηνεύεται ανάλογα με τον τύπο της μεταβλητής
 - ◆ Π.χ. integer, character, pointer, string, object reference, floating point number
- Δυναμική τιμή – μπορεί να αλλάζει κατά την εκτέλεση
 - ◆ Τότε το data object στο οποίο αποθηκεύεται η τιμή είναι και *μεταβλητή - variables*
- Στατική τιμή – δεν μπορεί να αλλάξει από το πρόγραμμα
 - ◆ Τότε το data object στο οποίο αποθηκεύεται η τιμή είναι *σταθερά - constant*



Τύποι και μεταβλητές (10/16)

■ Τύποι – *types*

- Ορίζουν σύνολα από τιμές και σχετικές λειτουργίες για τη δημιουργία, πρόσβαση και τροποποίηση αντίστοιχών τιμών
- Οι γλώσσες προσφέρουν ορισμένους προκαθορισμένους τύπους, π.χ. *boolean*, *integer*, *character*, καθώς και τις σχετικές λειτουργίες (π.χ. αριθμητικές πράξεις, εκχωρήσεις, κλπ)
- Συνήθως επιτρέπεται στους προγραμματιστές να ορίσουν νέους τύπους, π.χ., **class**, **struct**, **type**, ή και κατηγορίες τύπων , π.χ., **template**, **interface**



Τύποι και μεταβλητές (11/16)

■ Αντιστοίχιση τύπου – *type binding*

- Αφορά την απόδοση ενός τύπου σε μία μεταβλητή
- Μπορεί να γίνεται:
 - ◆ **Στατικά – static type binding**, δηλαδή κατά τη μεταγλώττιση (π.χ. Pascal, C, C++, Java, C#)
 - Είναι πιο γρήγορο, καθώς δεν απαιτούνται έλεγχοι τύπων κατά την εκτέλεση
 - ◆ **Δυναμικά – dynamic type binding**, με κάτι σαν assignment που αλλάζει τον τύπο κατά την εκτέλεση
 - Εάν η εκχώρηση μπορεί να γίνεται ελεύθερα απαιτεί type checking κατά την εκτέλεση καθώς και εφαρμογή των τελεστών που αφορούν το νέο τύπο



Τύποι και μεταβλητές (12/16)

■ Αναφορά σε τύπους – *referring to data types*

- Εμφανίζονται ως λέξεις κλειδιά, που χρησιμοποιούνται στον προσδιορισμό του τύπου μεταβλητών
 - ◆ π.χ. **int** x; **char*** x; **size_t** n;
 - ◆ Αυτό λέγεται και στατική αντιστοίχιση τύπου, **static typing**, γιατί ο τύπος κάθε μεταβλητής είναι γνωστός και παγιωμένος πριν από την εκτέλεση του προγράμματος.
 - ◆ Ειδικές λέξεις κλειδιά για το σύστημα τύπων - **int**, **struct**, **typedef**, **enum**, **class**, κλπ.
- Εμφανίζονται μόνο ως τιμές, ανάλογα με τον τύπο της τιμής
 - ◆ π.χ. x = **TRUE**; x = **3.14**;
 - ◆ Αυτό λέγεται δυναμική αντιστοίχιση τύπου, **dynamic typing**, καθώς ο τύπος κάθε μεταβλητής μπορεί να ποικίλει, καθοριζόμενος από τον τύπο της τιμής που κάθε φορά αποθηκεύεται σε αυτήν.



Τύποι και μεταβλητές (13/16)

■ Υποστήριξη τύπων – *type support* (1/2)

● *Strong typing - typed languages*

- ◆ Η γλώσσα προσφέρει μεθόδους ορισμού τύπων και επιβάλλει ελέγχους συμβατότητας μεταξύ τιμών, μεταβλητών και τύπων
- ◆ Η εκχώρηση τύπου σε μεταβλητές μπορεί να γίνεται:
 - στατικά, κατά τη μεταγλώττιση, που σημαίνει ότι ο τύπος μεταβλητής είναι αμετάβλητος – *static typing*, C, Pascal, C++, Java, C#
 - δυναμικά, κατά την εκτέλεση, ανάλογα με τον τύπο της τιμής που εκχωρείται σε μεταβλητές – *dynamic typing*, Python, Ruby, Dylan



Τύποι και μεταβλητές (14/16)

■ Υποστήριξη τύπων – *type support* (2/2)

● *No typing - untyped languages*

- ◆ Η γλώσσα προσφέρει απλούς εγγενείς τύπους ως τιμές και οι έλεγχοι συμβατότητας εφαρμόζονται μόνο για τιμές εγγενών τύπων κατά την εκτέλεση
 - *integer + function : error*
 - *integer + float : ok*
 - *function == function: ok*, κλπ
- ◆ Δεν υπάρχει ή έννοια ορισμού τύπων – no type definitions
- ◆ Χρησιμοποιούνται κάποιοι βασικοί τύποι για την προγραμματιστική εξομοίωση διαφόρων τύπων δεδομένων (όπως π.χ. θα φτιάχνατε μία λίστα με πίνακα στη C)
- ◆ Οι έλεγχοι συμβατότητας τύπων θα πρέπει να ενσωματώνονται από τον προγραμματιστή ως λογική του προγράμματος (δεν αυτοματοποιείται από τον compiler)
- ◆ Π.χ. Lua, alpha, Delta, Self



Τύποι και μεταβλητές (15/16)

■ Δήλωση μεταβλητών – *variable declaration* (1/2)

- Δήλωση πριν τη χρήση – *declaration before use*
 - ◆ Αυτό σημαίνει ότι πρέπει σε συγκεκριμένα σημεία του προγράμματος να δηλώνονται άμεσα οι μεταβλητές πριν χρησιμοποιηθούν
 - π.χ. **int i, n;** για static typing (δίνεται και ο τύπος)
 - π.χ. **var x,y,z;** για dynamic typing (χωρίς προσδιορισμό τύπου)
- Δήλωση με τη χρήση – *declaration by use*
 - ◆ Αυτό σημαίνει ότι κάθε νέο όνομα που χρησιμοποιείται αυτομάτως θεωρείται και νέα εμμέσως δηλωμένη μεταβλητή.
 - π.χ. **y = f(); x = g(y);** σημαίνει δήλωση του y και δήλωση του x, για την περίπτωση dynamic typing
 - π.χ. **(int x) = f((int y) = g());** κάτι αντίστοιχο με το προηγούμενο, αλλά για static typing, αν και δεν είναι τόσο εύχρηστο, όπως άλλωστε είναι προφανές



Τύποι και μεταβλητές (16/16)

■ Δήλωση μεταβλητών – *variable declaration* (2/2)

- Οι διαφορετικές τακτικές δεν πρέπει να σας προβληματίζουν
- Στη C έπρεπε να δηλώσετε τις τοπικές μεταβλητές στην αρχή του block, ανεξάρτητα του σημείου χρήσης
- Στη C++ μπορείτε να δηλώνετε τις μεταβλητές όπου θέλετε, αλλά πριν το σημείο χρήσης
 - ◆ Υπάρχουν οδηγίες για δήλωση όσο πιο κοντά στη χρήση γίνεται
- Η C++ είναι πιο κοντά στην τακτική «δήλωση με τη χρήση», απλώς την υποστηρίζει ως «δήλωση πολύ κοντά στη χρήση».
 - ◆ Ωστόσο, θα μπορούσε εύκολα δημιουργηθεί παραλλαγή της C++ ώστε η δήλωση να εφάπτεται της χρήσης απόλυτα



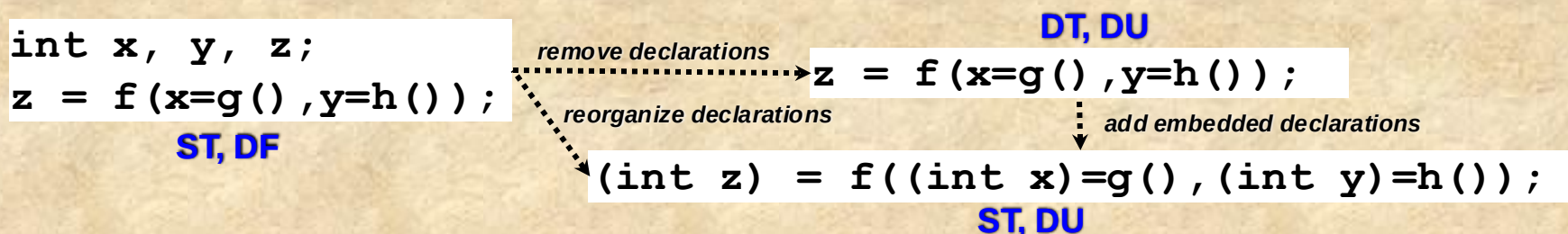
Περιεχόμενα

- Ο ρόλος του μεταγλωττιστή και οι φάσεις μετάφρασης
- Γλώσσες προγραμματισμού
- Τύποι και μεταβλητές
- *Static and dynamic typing*
- Κύκλοι επεξεργασίας στη μετάφραση
- Διερμηνείς και μεταφραστές
- Παραγωγή κώδικα για πραγματικές ή εικονικές μηχανές
- Οι μεταγλωττιστές υπάρχουν παντού



Static and dynamic typing (1/6)

- Τι αξία έχουν όλες αυτές οι εναλλακτικές μορφές για τον προγραμματιστή; υπάρχει κάποια ισοδυναμία μεταξύ τους;
 - ST=static typing, DT= dynamic typing
 - DF=declaration before use, DU= declaration by use



DT, DU

```
x="hello";  
x=true;  
x = 3.14;
```

DT, DF

```
var x,y,z;  
x="hello";  
x=true;  
x = 3.14;
```

ST, DF

```
char* x;  
x="hello";  
x=true;  
x = 3.14;
```

ST, DU

```
(char*) x="hello";  
x=true;  
x = 3.14;
```



Static and dynamic typing (2/6)

```
int x = y-N;
while (x--) {
    //code
}
int w=sqrt(x);
int z=sqr(x)+sqr(w);
```

```
while ((int x = y-N)--) {
    // code
}

(int z) = sqr(x)+sqr(int w=sqrt(x));
```

Οι διαφορές μεταξύ των δύο εκδόσεων ?

- Η δεύτερη απαιτεί λιγότερες ξεχωριστές δηλώσεις
- Η πρώτη είναι πιο «εύπεπτη» για τον προγραμματιστή
- Η δεύτερη δεν υποστηρίζει πολλαπλές δηλώσεις
- Η δεύτερη δεν ορίζει με σαφήνεια τη σειρά αρχικοποιήσεων.

...

Η γλώσσα προγραμματισμού επηρεάζει σημαντικά την τακτική ανάπτυξης που υιοθετείτε, την ωριμότητα προγραμματισμού, αλλά και τη δυνατότητα να προχωρήσετε σε πολύ εξελιγμένα προγραμματιστικά σχήματα και μεγάλες εφαρμογές

- Assembly
- Pascal, C
- C++
- C++ with templates
- Προηγμένες γλώσσες ειδικού / γενικού σκοπού



Static and dynamic typing (3/6)

- Όπως φαίνεται, με dynamic typing αποφεύγονται οι προσδιορισμοί τύπων στις μεταβλητές
 - Αυτό κάνει τη γλώσσα πιο «εύκολη», τα προγράμματα πιο μικρά και πιο «ευέλικτα»
 - Αλλά καθώς δεν μπορούμε στη διαδικασία μεταγλώττισης να εφαρμόσουμε έλεγχο τύπων είναι πιθανό να εμφανιστούν ασυμφωνίες τύπων κατά την εκτέλεση
 - ◆ Δηλ. *αρκετά compile-errors των static typing γλωσσών γίνονται run-time errors των dynamic typing γλωσσών*
 - ◆ Αυτό λέγεται και *lack of type safety*
- Παρόμοια ευελιξία θεωρητικά επιτυγχάνεται και με declaration by use, καθώς το πρόγραμμα γίνεται πιο лакωνικό, αφού δεν χρειάζονται δηλώσεις
 - Αλλά χρειάζεται να μπορούμε να ξεχωρίζουμε την εμβέλεια των μεταβλητών, και να λάβουμε υπόψη και την αναγνωσιμότητα του προγράμματος



Static and dynamic typing (4/6)

Dynamic typing Javascript-style

```
function sqrrpair (x,y) { return x*x + y*y; }  
z = sqrrpair(10, 20.5);  
z = sqrrpair(3.14, 1.9);
```

Static typing C-style

```
#define RESULT x*x + y*y
```

```
float sqrrpair_intdouble (int x, double y)    { return RESULT; }  
double sqrrpair_doubleint (double x, int y)   { return RESULT; }  
int sqrrpair_intint (int x, int y)            { return RESULT; }
```

```
double z = sqrrpair_intdouble (10, 20.5);  
z = sqrrpair_doubleint(3.14, 10);
```

Η λύση είναι ο γενικότερος τύπος, αλλά δεν ορίζεται «καλώς» πάντα

```
double sqrrpair (double x, double y) { return RESULT; }
```

Για διαφορετικούς τύπους αναγκάζομαστε να έχουμε διαφορετικές υλοποιήσεις, το οποίο μπορεί να μας οδηγήσει σε διαφορετικές συναρτήσεις. Ωστόσο, ακόμη και με την επιλογή του «γενικότερου τύπου», υπάρχουν περιπτώσεις που απλά δεν υπάρχει γενικότερος τύπος.

Static and dynamic typing (5/6)

- Η χρήση δυναμικών γλωσσών παρέχει εκφραστική οικονομία λόγω μη αναγκαίας συμμόρφωσης σε type και function signatures κατά τη μεταγλώττιση
- Π.χ., στη C δεν μπορούμε να κατασκευάσουμε: **συνάρτηση f που έχει δύο ορίσματα, μία συνάρτηση g και μία τιμή x , και να επιστρέφει το $g(x)$, $\forall g$ και x .**
- Στη C++ μπορούμε να προσπαθήσουμε, αλλά θα πρέπει να αναφέρουμε αναγκαστικά τον επιστρεφόμενο τύπο.

```
template <class Y> class f {  
    public:  
    template <class G, class X>  
    const Y operator() (const G& g, const X& x) {  
        return g(x);  
    }  
};  
  
int g (int x) { return x+1; }  
y = f<int>() (g, 1);
```

Η υλοποίηση στη C++ πάσχει από χρήση μη αλγοριθμικού κώδικα ώστε να γενικεύεται η «κάψουλα» σε διάφορους τύπους. Αλλά και πάλι δεν έχει την ευκολότερη χρήση

```
function f(g,x) { return g(x); }
```

Η υλοποίηση στην alpha είναι τόσο απλή όσο θα ήθελε κανείς. Αρκεί κατά την εκτέλεση το g να είναι συνάρτηση βέβαια...



Static and dynamic typing (6/6)

Dynamic typing style – μεγάλη ευελιξία, αλλά διαχείριση τύπων από τον προγραμματιστή!

```
function sqrrpair (x,y) {  
    if (typeof(x) == "string") x = stringtonumber(x) ;  
    if (typeof(y) == "string") y = stringtonumber(y) ;  
    return x*x + y*y;  
}  
x = sqrrpair(10, "30");
```

Declaration by use – ευκολία, αλλά χρειάζεται διαχωρισμός εμβέλειας!

```
i = 10;  
function sum1 (p, n) {  
    for ( i = s = 0; i < n; ++i ) // To i είναι το global by default  
        s += p[i];  
}  
function sum2 (p, n) {  
    for ( local i = local s = 0; i < n; ++i ) // Επιβάλλουμε νέο local i  
        s += p[i];  
}
```

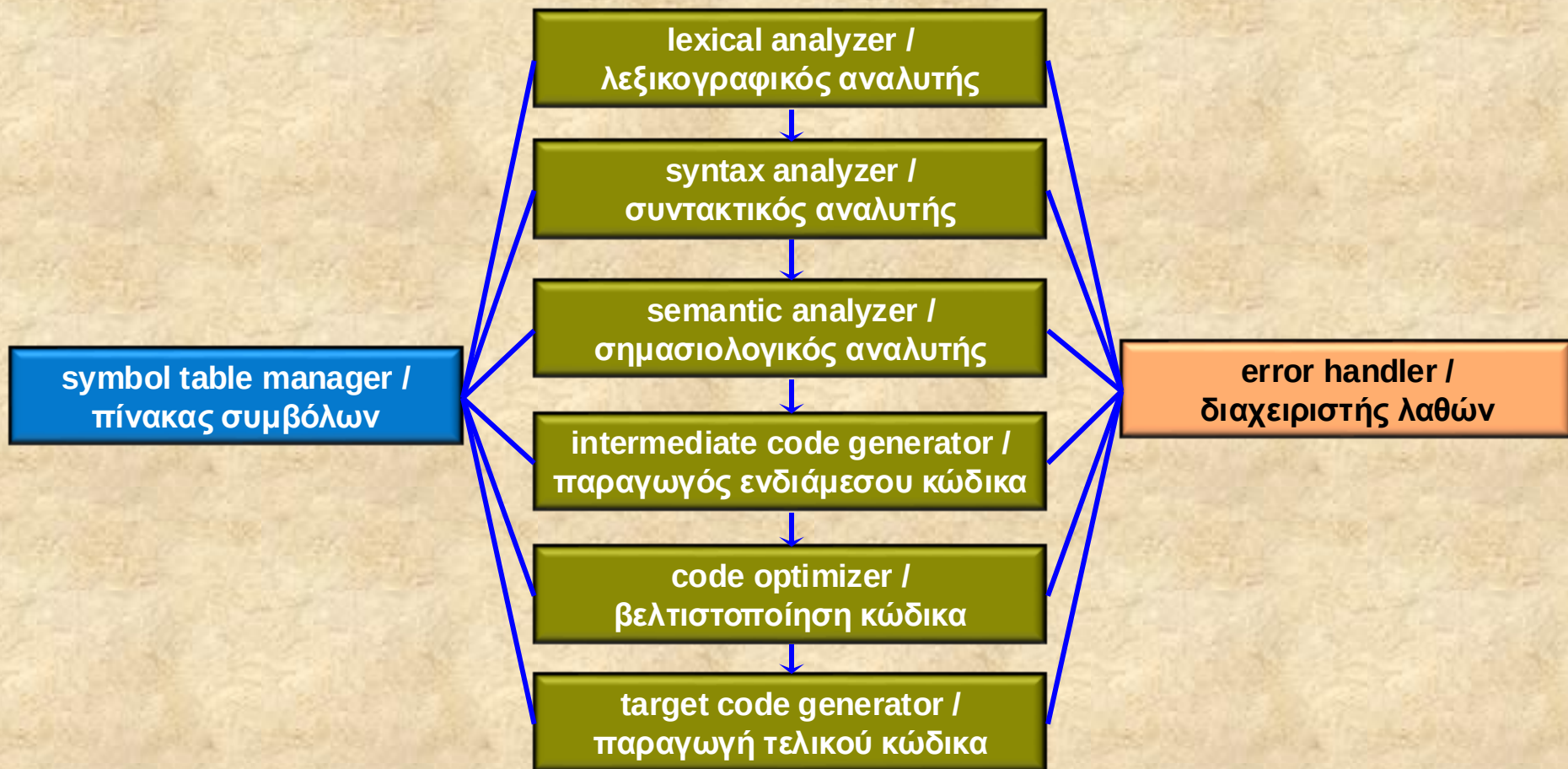


Περιεχόμενα

- Ο ρόλος του μεταγλωττιστή και οι φάσεις μετάφρασης
- Γλώσσες προγραμματισμού
- Τύποι και μεταβλητές
- Static and dynamic typing
- *Κύκλοι επεξεργασίας στη μετάφραση*
- Διερμηνείς και μεταφραστές
- Παραγωγή κώδικα για πραγματικές ή εικονικές μηχανές
- Οι μεταγλωττιστές υπάρχουν παντού

Κύκλοι επεξεργασίας (1/16)

- Η γενική αρχιτεκτονική ενός μεταγλωττιστή, τεμαχισμένη σε τμήματα με συγκεκριμένο λειτουργικό ρόλο το καθένα



Κύκλοι επεξεργασίας (2/16)

■ Παράδειγμα (1/3)

`x = 3.14 * y + z;`

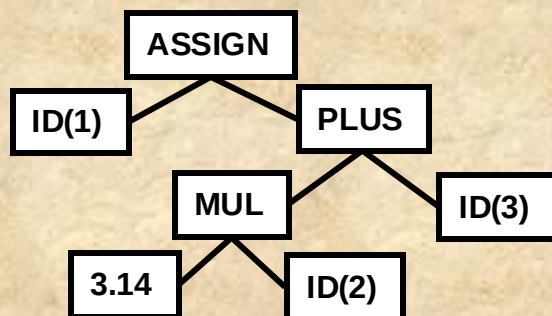
lexical analyzer

symbol table

ID(1)	ASSIGN	3.14	MUL	ID(2)	PLUS	ID(3)
-------	--------	------	-----	-------	------	-------

ID	NAME	DATA
1	x	
2	y	
3	z	
		

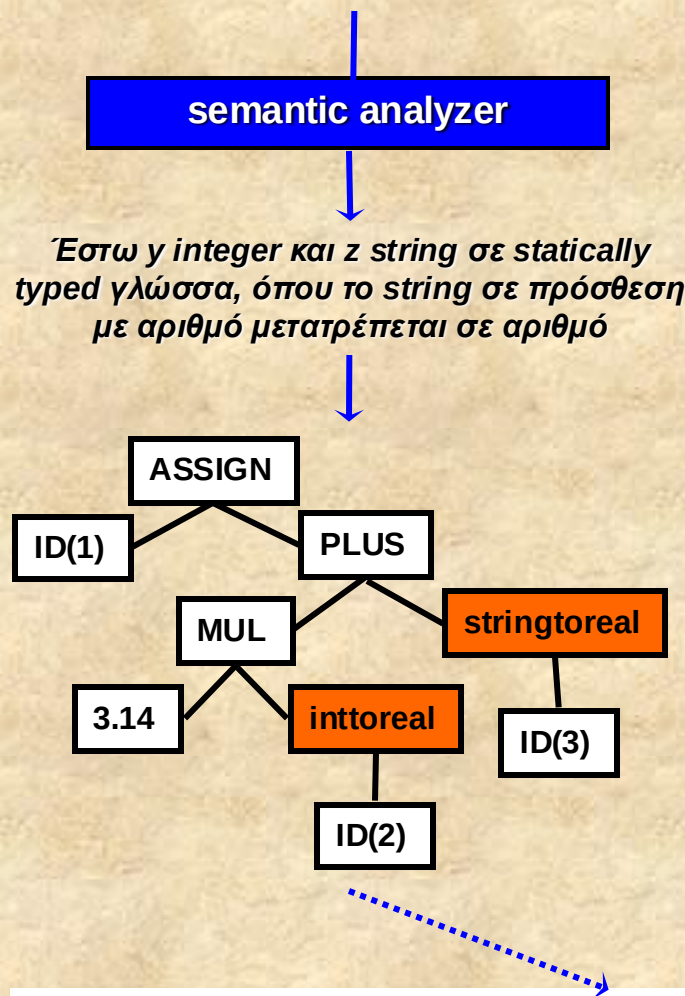
syntax analyzer



Ο lexical analyzer μετατρέπει την ακολουθία χαρακτήρων σε μία ακολουθία «στοιχείων» αντιστοιχώντας σύμβολα - κλειδιά (π.χ. *) σε τελεστές της γλώσσας (π.χ. MUL). Ταυτόχρονα, για τα αναγνωριστικά ονόματα δημιουργεί εγγραφές στον πίνακα συμβόλων, ενώ στην παραγόμενη ακολουθία στοιχείων, για κάθε όνομα βάζει την κατάλληλη αναφορά στον πίνακα συμβόλων, δηλ. $ID(j)$

Ο syntax analyzer μετατρέπει μία σειριακή ακολουθία «στοιχείων» σε μία οργανωμένη ιεραρχική (δενδρική) δομή. Η δομή αυτή αντιπροσωπεύει την συντακτική / γραμματική οργάνωση της ακολουθίας συμβόλων, ενσωματώνοντας ταυτόχρονα και πληροφορία για την ακολουθία υπολογισμού (δηλ. εκτέλεση προγράμματος) των εκφράσεων και εντολών που περιέχονται. Π.χ. ένας τρόπος «εκτέλεσης» της ιεραρχικής δομής θα ήταν: «εφάρμοσε την πράξη υπολογίζοντας το αριστερότερο όρισμα πρώτα και έπειτα το δεξιότερο».

Κύκλοι επεξεργασίας (3/16)



•Ο semantic analyzer εφαρμόζει κανόνες σημασιολογικού ελέγχου στην γραμματική δομή όπως αυτοί ορίζονται στην τεκμηρίωση της γλώσσας.

•Οι πιο συνηθισμένοι είναι οι «κανόνες συμβατότητας και μετατροπής τύπων» - type checking / conversion rules, οι οποίοι ισχύουν κυρίως για γλώσσες με static typing (ή αλλιώς και strongly typed γλώσσες).

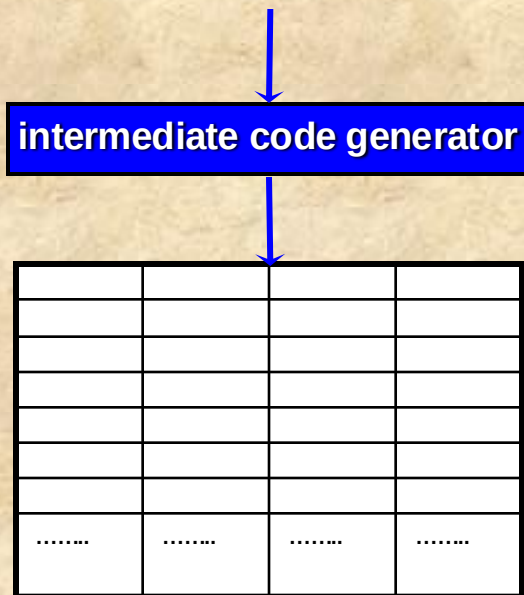
•Από αυτή την ανάλυση μπορεί να επεκταθεί ελαφρώς το δέντρο εισόδου, καθώς και να έχουμε λάθη λόγω ασυμφωνίας τύπων.

•Η τροποποιημένη ιεραρχική δομή για την εντολή του προγράμματος τώρα αντιστοιχεί εσωτερικά σε μία «επαυξημένη» εντολή που περιέχει τόσο τις απαραίτητες μετατροπές όσο και τις αναφορές στις εγγραφές του πίνακα συμβόλων.

$ID(1):x = 3.14 * \text{inttoreal}(ID(2):y) + \text{stringtoreal}(ID(3):z)$



Κύκλοι επεξεργασίας (4/16)



Η δουλειά του παραγωγού ενδιάμεσου κώδικα φαίνεται να είναι η μετατροπή του προηγούμενου δέντρου σε μία σειρά κατάλληλων εντολών αυτής της «ενδιάμεσης μηχανής».

Κώδικας εντολής	Διεύθυνση ορίσματος 1	Διεύθυνση ορίσματος 2	Διεύθυνση αποτελέσματος
-----------------	-----------------------	-----------------------	-------------------------

•Ο ενδιάμεσος κώδικας είναι μία μορφή κώδικα «γενικής / αφηρημένης μηχανής» (επεξεργαστή), με δύο βασικές ιδιότητες: εύκολο να παραχθεί, εύκολο να μεταφραστεί σε συγκεκριμένη γλώσσα μηχανής. **Μοιάζει πολύ με assembly.**

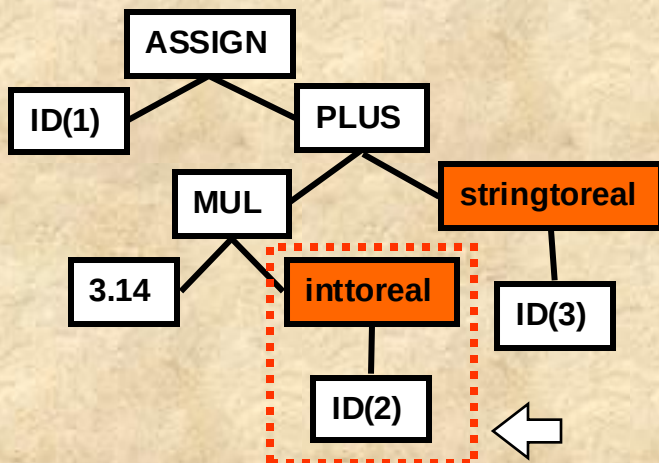
•Συνήθως υιοθετούμε ενδιάμεσο κώδικα με **εντολές τριών πεδίων διευθύνσεων – three address code**: (α) αποτέλεσμα, (β) τελεστής και (γ) δύο ορίσματα. Οι εντολές του ενδιάμεσου κώδικα ονομάζονται και **quads – τετράδες**.

•Για την παραγωγή ενδιάμεσου κώδικα για εκφράσεις, ο μεταγλωττιστής θα πρέπει να παράγει πολλές τέτοιες εντολές, χρησιμοποιώντας προσωρινές μεταβλητές (δηλ. εμμέσως δηλωμένες μεταβλητές) για την αποθήκευση ενδιάμεσων αποτελεσμάτων.



Κύκλοι επεξεργασίας (5/16)

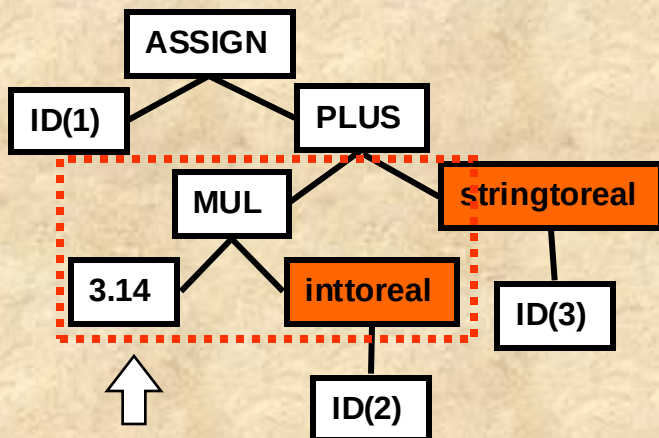
Αυτός είναι ένας απλός αναδρομικός αλγόριθμος με διάσχιση DFS (depth first search) για την παραγωγή των εντολών ενδιάμεσου κώδικα.



```
temp1 = inttoreal(ID(2))
```

```
produce_intermediate_code(node)
begin
  if node.type = Variable then
    return node.variable;
  if node.type = Constant then
    return node.constant;
  if node.type = Conversion then
    begin
      right = produce_intermediate(node.right);
      temp = new_temp_variable();
      generate(temp, "=", node.conversion, right);
      return temp;
    end
  if node.type = Operator then
    begin
      left = produce_intermediate(node.left);
      right = produce_intermediate(node.right);
      temp = new_temp_variable();
      generate(temp, node.operator, left, right);
      return temp;
    end
end
```

Κύκλοι επεξεργασίας (6/16)



```
temp1 = inttoreal(ID(2))
temp2 = 3.14 * temp1
```

```
produce_intermediate_code(node)
```

```
begin
```

```
if node.type = Variable then
```

```
    return node.variable;
```

```
if node.type = Constant then
```

```
    return node.constant;
```

```
if node.type = Conversion then
```

```
begin
```

```
    right = produce_intermediate(node.right);
```

```
    temp = new_temp_variable();
```

```
    generate(temp, "=", node.conversion, right);
```

```
    return temp;
```

```
end
```

```
if node.type = Operator then
```

```
begin
```

```
    left = produce_intermediate(node.left);
```

```
    right = produce_intermediate(node.right);
```

```
    temp = new_temp_variable();
```

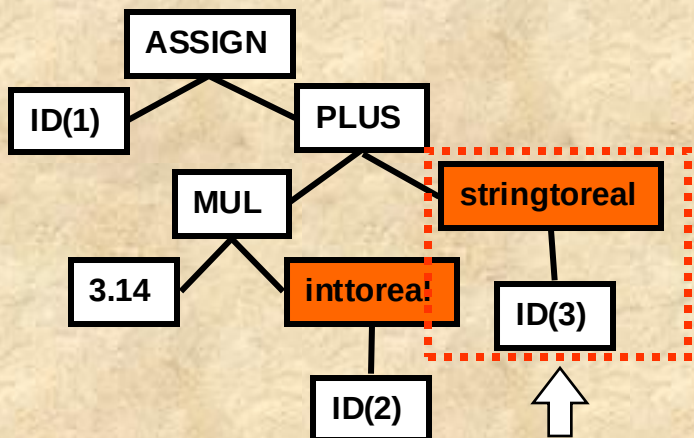
```
    generate(temp, node.operator, left, right);
```

```
    return temp;
```

```
end
```

```
end
```

Κύκλοι επεξεργασίας (7/16)



```

temp1 = inttoreal(ID(2))
temp2 = 3.14 * temp1
temp3 = stringtoreal(ID(3))

```

```
produce_intermediate_code(node)
```

```
begin
```

```
if node.type = Variable then
```

```
    return node.variable;
```

```
if node.type = Constant then
```

```
    return node.constant;
```

```
if node.type = Conversion then
```

```
begin
```

```
    right = produce_intermediate(node.right);
```

```
    temp = new_temp_variable();
```

```
    generate(temp, "=", node.conversion, right);
```

```
    return temp;
```

```
end
```

```
if node.type = Operator then
```

```
begin
```

```
    left = produce_intermediate(node.left);
```

```
    right = produce_intermediate(node.right);
```

```
    temp = new_temp_variable();
```

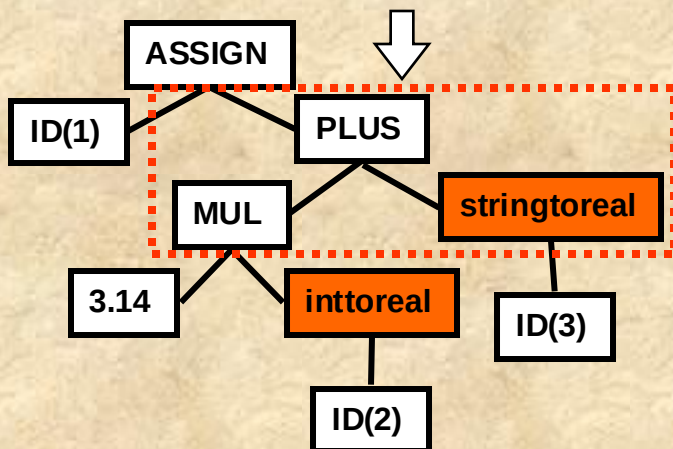
```
    generate(temp, node.operator, left, right);
```

```
    return temp;
```

```
end
```

```
end
```


Κύκλοι επεξεργασίας (8/16)



```

temp1 = inttoreal(ID(2))
temp2 = 3.14 * temp1
temp3 = stringtoreal(ID(3))
temp4 = temp2 + temp3
  
```

```
produce_intermediate_code(node)
```

```
begin
```

```
if node.type = Variable then
```

```
    return node.variable;
```

```
if node.type = Constant then
```

```
    return node.constant;
```

```
if node.type = Conversion then
```

```
begin
```

```
    right = produce_intermediate(node.right);
```

```
    temp = new_temp_variable();
```

```
    generate(temp, "=", node.conversion, right);
```

```
    return temp;
```

```
end
```

```
if node.type = Operator then
```

```
begin
```

```
    left = produce_intermediate(node.left);
```

```
    right = produce_intermediate(node.right);
```

```
    temp = new_temp_variable();
```

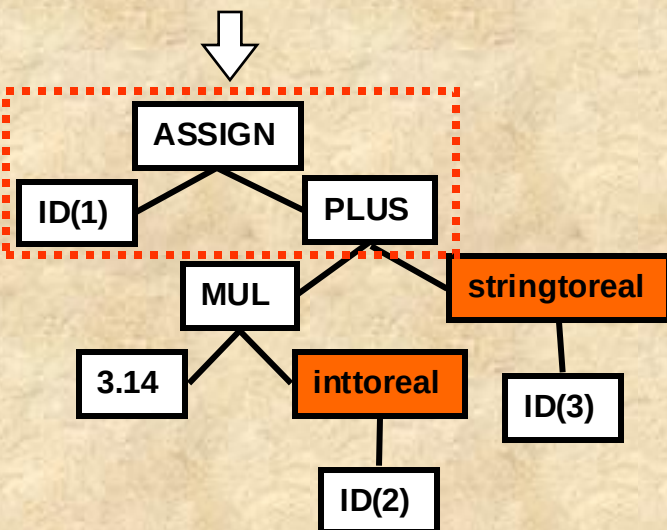
```
    generate(temp, node.operator, left, right);
```

```
    return temp;
```

```
end
```

```
end
```

Κύκλοι επεξεργασίας (9/16)



```

temp1 = inttoreal(ID(2))
temp2 = 3.14 * temp1
temp3 = strinttoreal(ID(3))
temp4 = temp2 + temp3
ID(1) = temp4
  
```

produce_intermediate_code(node)

begin

if *node.type* = Variable **then**

return *node.variable*;

if *node.type* = Constant **then**

return *node.constant*;

if *node.type* = Conversion **then**

begin

right = *produce_intermediate*(*node.right*);

temp = *new_temp_variable*();

generate(*temp*, "=", *node.conversion*, *right*);

return *temp*;

end

if *node.type* = Operator **then**

begin

left = *produce_intermediate*(*node.left*);

right = *produce_intermediate*(*node.right*);

temp = *new_temp_variable*();

generate(*temp*, *node.operator*, *left*, *right*);

return *temp*;

end

end



Κύκλοι επεξεργασίας (10/16)

intermediate code generator

```
temp1 = inttoreal(ID(2))  
temp2 = 3.14 * temp1  
temp3 = strinttoreal(ID(3))  
temp4 = temp2 + temp3  
ID(1) = temp4
```

intermediate code optimizer

```
temp1 = inttoreal(ID(2))  
temp2 = 3.14 * temp1  
temp3 = strinttoreal(ID(3))  
ID(1) = temp2 + temp3
```

• Η βελτιστοποίηση του ενδιάμεσου κώδικα έχει μόνο ένα σκοπό: να «μετατρέψει» τον ενδιάμεσο κώδικα ώστε γρηγορότερος κώδικας μηχανής να μπορεί να παραχθεί.

• Συνήθως η έννοια του «μετατρέψει» συνεπάγεται «να ελαττώσει τις εντολές», βασισμένη στον κανόνα ότι εάν αφαιρέσουμε εντολές ενδιάμεσου κώδικα αφαιρούμε ουσιαστικά εντολές κώδικα μηχανής, που σημαίνει ότι ο παραγόμενος κώδικας μηχανής «τρέχει» γρηγορότερα.

• Μία πολύ απλή τακτική είναι να αφαιρεθούν ενδιάμεσες εντολές αποθήκευσης αποτελεσμάτων, όπως πράττουμε και στο διπλανό παράδειγμα.

Κύκλοι επεξεργασίας (11/16)

target code generation

PUSH	ID (2)
CALLLIB	inttoreal
GETRET	R1
MULF	3.14, R1
PUSH	ID (3)
CALLLIB	strinttoreal
GETRET	R2
ADDF	R2, R1
MOVF	R1, ID (1)

- Η τελευταία φάση στην διαδικασία μεταγλώττισης είναι η παραγωγή κώδικα στη γλώσσα προορισμού.
- Σε πολλούς μεταγλωττιστές κάτι τέτοιο είναι είτε γλώσσα συγκεκριμένης μηχανής (π.χ. Pentium assembly) ή γλώσσα εικονικής μηχανής (π.χ. Java byte code).
- Στη διαδικασία αυτή συγκεκριμένες θέσεις μνήμης επιλέγονται για τις μεταβλητές του προγράμματος, ενώ οι εντολές του ενδιαμέσου κώδικα μεταφράζονται σε αλληλουχία εντολών μηχανής.
- Μία πολύ σημαντική εργασία σε αυτή τη φάση είναι η αντιστοίχιση των μεταβλητών σε καταχωρητές – *variable to register allocation*.

R1: temp1

temp1 = inttoreal (ID (2))

R1: temp2

temp2 = 3.14 * temp1

R2: temp3

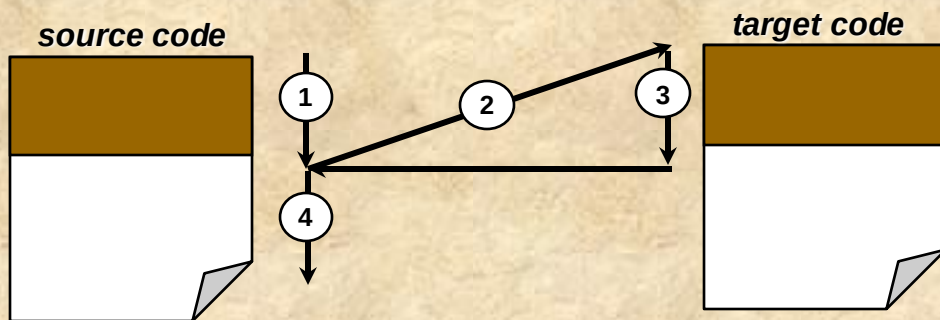
temp3 = strinttoreal (ID (3))

ID (1) = temp2 + temp3

Σε αυτό το παράδειγμα, η γλώσσα μηχανής έχει εντολές του τύπου $\langle Op \rangle \langle src \rangle \langle dest \rangle$ που αντιστοιχούν στην πράξη $\langle dest \rangle = \langle src \rangle \text{ op } \langle dest \rangle$. Π.χ. η **MULF 3.14, R1** σημαίνει $R1 = 3.14 * R1$. Η εντολή MOVF φορτώνει το περιεχόμενο της θέσης μνήμης μίας μεταβλητής σε έναν καταχωρητή και αντίστροφα.

Κύκλοι επεξεργασίας (12/16)

- Οι μεταγλωττιστές ενός κύκλου, η ενός περάσματος – *single pass compilers*, παράγουν τελικό κώδικα ταυτόχρονα με την ανάγνωση του πηγαίου κώδικα



- 1: Λεξιλογαφική ανάλυση ενός στοιχείου,
- 2: Συντακτική, σημασιολογική ανάλυση, ενδιάμεσος κώδικας
- 3: Παραγωγή τελικού κώδικα
- 4: Ανάγνωση επόμενου στοιχείου

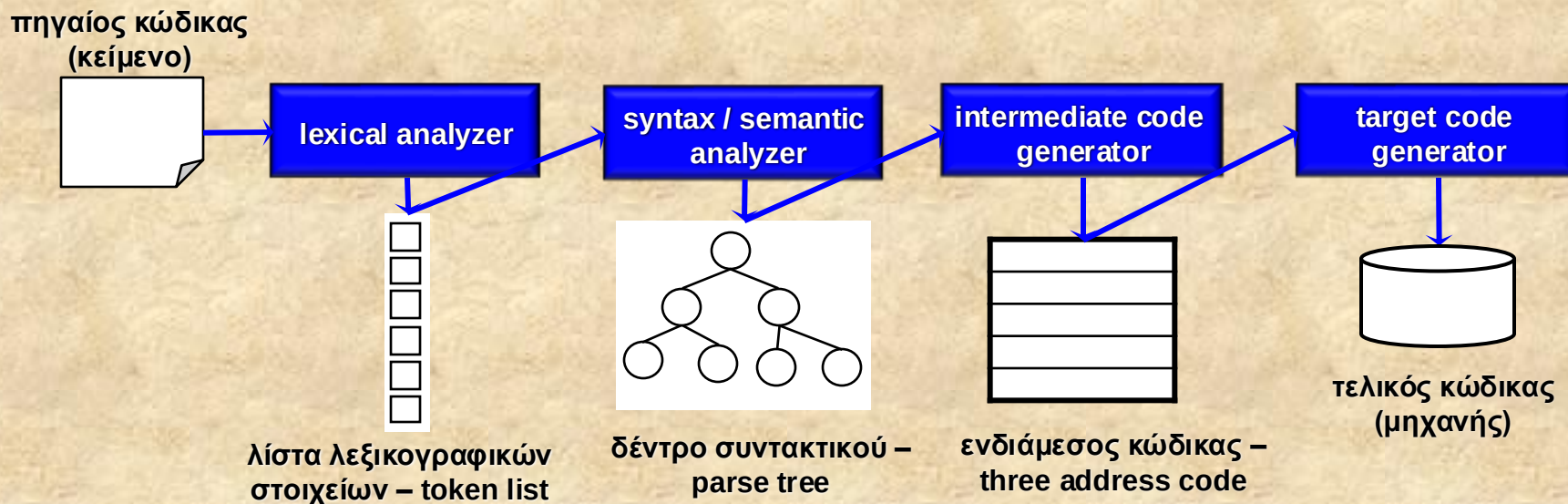
•Η παραγωγή τελικού κώδικα κατά τη διάρκεια της ανάγνωσης του πηγαίου κώδικα και της παραγωγής ενδιάμεσου κώδικα δεν είναι εύκολο πρόβλημα.

•Οι κύριες δυσκολίες οφείλονται στο φαινόμενο που είναι γνωστό ως «εμπρόσθιες εξαρτήσεις» - forward dependencies, κατά τις οποίες πρέπει να παραχθεί κώδικας με κάποιες εντολές JUMP / BRANCH / GOTO των οποίων η διεύθυνση αφορά εντολές κώδικα που θα παραχθούν αργότερα, με την μεταγλώττιση πηγαίου κώδικα που έπεται του παρόντος σημείου.

•Σχεδόν πάντα κάτι τέτοιο λύνεται αφήνοντας «κενές» τιμές και κρατώντας σε μία λίστα προηγούμενες εντολές τις οποίες «επιδιορθώνουμε αργότερα» με μία τεχνική γνωστή ως back-patching.

Κύκλοι επεξεργασίας (13/16)

- Οι μεταγλωττιστές πολλών περασμάτων – *multi pass compilers* πρακτικά αποτελούνται από ξεχωριστά υποσυστήματα για κάθε φάση



Συνήθως χρησιμοποιούμε μεταγλωττιστές πολλαπλών περασμάτων για τους εξής λόγους: (α) η μνήμη είναι λίγη (δεν υφίσταται σήμερα), (β) η γλώσσα είναι πολύ πολύπλοκη, (γ) η μεταφορά σε άλλες πλατφόρμες είναι απαραίτητη. Ωστόσο, η αναγωγή ενός single-pass compiler σε multi-pass compiler σημαίνει και ενδιάμεση αποθήκευση των παραγώγων κάθε φάσης, το οποίο απαιτεί επιπλέον κώδικα.

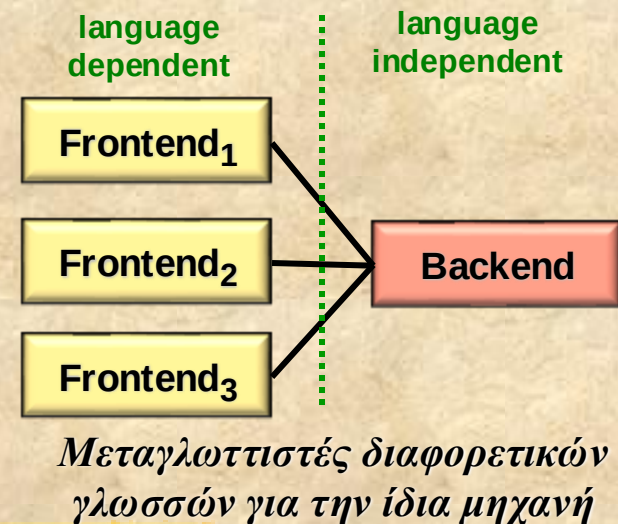
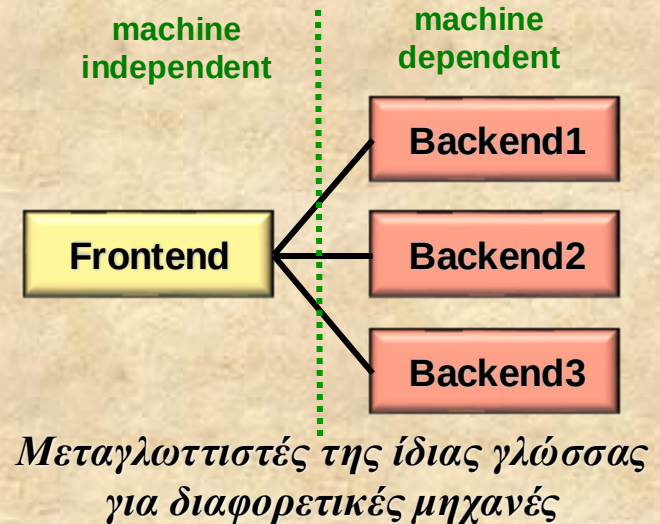


Κύκλοι επεξεργασίας (14/16)

- Ένας compiler διαχωρίζεται γενικά σε δύο κατηγορίες τμημάτων:
 - **frontend**, περιλαμβάνει τα τμήματα που είναι εξαρτημένα από τη γλώσσα πηγής, ενώ είναι αρκετά ανεξάρτητα από τη γλώσσα προορισμού
 - ◆ αυτά συνήθως είναι : lexical analyzer, symbol table, syntactic analyzer, intermediate code generator, intermediate code optimizer
 - **backend**, περιλαμβάνοντας τα τμήματα τα οποία εξαρτώνται από τη γλώσσα προορισμού
 - ◆ αυτά συνήθως είναι: ειδικά χαρακτηριστικά του symbol table, target code generator, target code optimizer

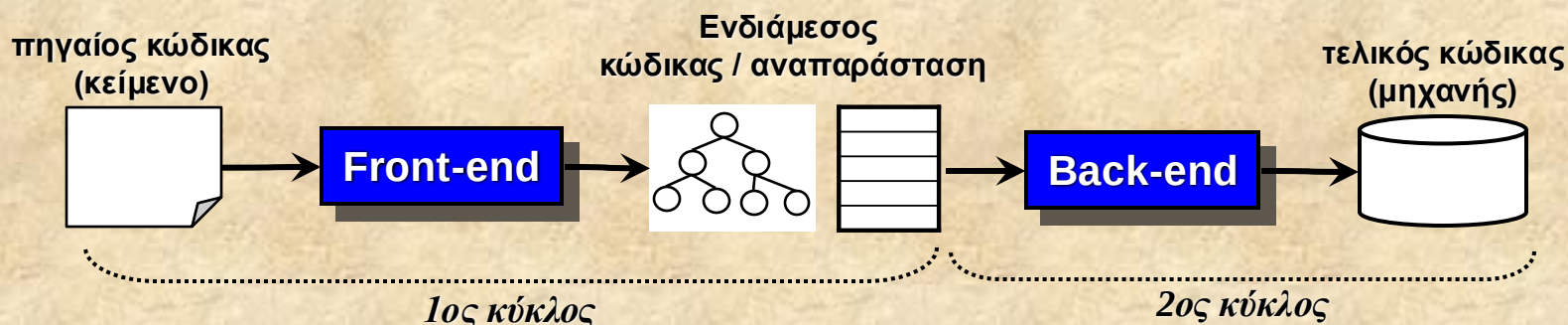
Κύκλοι επεξεργασίας (15/16)

- Είναι πολύ συνηθισμένο να λαμβάνουμε το frontend ενός compiler και να ξανά-υλοποιούμε το backend για την ίδια γλώσσα και διαφορετική μηχανή
- Επίσης, ένας ιδιαίτερα απαιτητικός στόχος είναι η μεταγλώττιση διαφορετικών γλωσσών χρησιμοποιώντας τον ίδιο τύπο ενδιάμεσου κώδικα
 - με χρήση του ίδιου backend ώστε να επιτυγχάνουμε στο τέλος πολλούς μεταγλωττιστές για την ίδια μηχανή
 - με εξαίρεση το .NET, λίγη πρόοδος έχει επιτελεστεί προς αυτή την κατεύθυνση



Κύκλοι επεξεργασίας (16/16)

- Οι μεταγλωττιστές δύο κύκλων – *two cycle compilers*, είναι οι πιο συνηθισμένοι σήμερα. Διακρίνονται σε δύο ξεχωριστές διαδικασίες



- **Πλεονεκτήματα:** καλύτερο portability, πολλοί συνδυασμοί με διαφορετικά front-ends και back-ends είναι εφικτοί
- **Μειονεκτήματα:** πιο αργή μετάφραση, χρειάζεται περισσότερη μνήμη (η μνήμη του 1ου κύκλου δεν απελευθερώνεται παρά μόνο με το πέρας του 2ου κύκλου).



Περιεχόμενα

- Ο ρόλος του μεταγλωττιστή και οι φάσεις μετάφρασης
- Γλώσσες προγραμματισμού
- Τύποι και μεταβλητές
- Static and dynamic typing
- Κύκλοι επεξεργασίας στη μετάφραση
- *Διερμηνείς και μεταφραστές*
- Παραγωγή κώδικα για πραγματικές ή εικονικές μηχανές
- Οι μεταγλωττιστές υπάρχουν παντού



Διερμηνείς και μεταφραστές (1/3)

- Ο διερμηνέας (interpreter) εφαρμόζει συντακτική ανάλυση και εκτελεί απευθείας τις εντολές του προγράμματος
 - Αυτό σημαίνει ότι τα όποια ορθογραφικά ή συντακτικά λάθη εντοπίζονται κατά την εκτέλεση – runtime
 - ◆ Άρα ο προγραμματιστής πρέπει να εφαρμόσει εξαντλητικό testing για να εξασφαλίσει ότι όλα τα πιθανά μονοπάτια εκτέλεσης έχουν δοκιμαστεί, αφού μπορούν να εμπεριέχουν και τέτοια λάθη
- Διερμηνείς χρειάζονται σε όλες τις περιπτώσεις που είναι πιο ευέλικτο να έχουμε «εκτελέσιμο source code», χωρίς να περιοριζόμαστε μόνο σε γλώσσες προγραμματισμού.
 - Java Script, Perl, Python
 - HTML, XML
 - Shell script για shell programming
 - Configuration files (.INI, .rc, .cfg, κλπ)
 - Data parsers and loaders

Διερμηνείς και μεταφραστές (2/3)

- Ο μεταγλωττιστής (compiler) είναι η ειδική περίπτωση ενός μεταφραστή όταν:
 - η γλώσσα εισόδου είναι μία γλώσσα προγραμματισμού υψηλού επιπέδου (Pascal, C++, C, Java, κλπ)
 - ενώ ο παραγόμενος τελικός κώδικας είναι σε εκτελέσιμη μορφή κάποιας μηχανής
 - ◆ Είτε πραγματικής, δηλ. συγκεκριμένου επεξεργαστή όπως Pentium, MIPS, κλπ
 - ◆ Είτε εικονικής, δηλ. που εκτελείται από υλοποίηση της εικονικής μηχανής (virtual machine), όπως π.χ. το Java Virtual Machine (JVM) για Java Byte Code ή .NET runtime για CLR code
- Η κατασκευή ενός compiler είναι αρκετά απαιτητική, με πολυπλοκότητα εξαρτώμενη τόσο από την γλώσσα εισόδου (συντακτικό και σημασιολογία) όσο και από το είδος του κώδικα μηχανής (δηλ. της μηχανής)



Διερμηνείς και μεταφραστές (3/3)

- Ο μεταφραστής είναι η ευρύτερη οικογένεια των συστημάτων τα οποία δέχονται προτάσεις μίας γλώσσας εισόδου **A** και παράγουν προτάσεις μίας γλώσσας εξόδου **B**
 - Σημειώνεται ότι οι προτάσεις της γλώσσας εξόδου **B** μπορούν κάλλιστα να περιέχουν ενέργειες που εκτελούνται άμεσα
 - ◆ Π.χ. ένας OS shell έχει ως γλώσσα εισόδου shell scripts και ως γλώσσα εξόδου shell commands (που εκτελούνται)
 - Η τέχνη κατασκευής μεταφραστών δεν περιορίζεται μόνο σε programming language compilers ή interpreters
 - ◆ αλλά μπορεί να χρησιμοποιηθεί για μία πληθώρα εφαρμογών, προσφέροντας πολύτιμους αυτοματισμούς και λύσεις



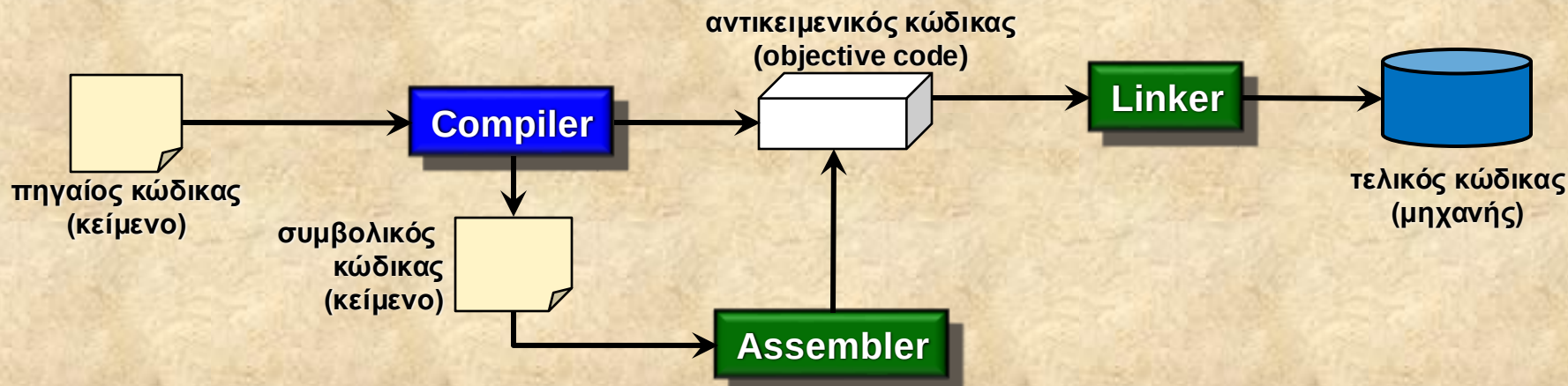
Περιεχόμενα

- Ο ρόλος του μεταγλωττιστή και οι φάσεις μετάφρασης
- Γλώσσες προγραμματισμού
- Τύποι και μεταβλητές
- Static and dynamic typing
- Κύκλοι επεξεργασίας στη μετάφραση
- Διερμηνείς και μεταφραστές
- *Παραγωγή κώδικα για πραγματικές ή εικονικές μηχανές*
- Οι μεταγλωττιστές υπάρχουν παντού



Κώδικας για πραγματικές ή εικονικές μηχανές (1/4)

- Όταν παράγεται κώδικας για πραγματικές μηχανές αυτός είναι σε μορφή:
 - αντικειμενική (*objective code*) που μπορεί να συνδεθεί απευθείας από τον linker για τη συρραφή του εκτελέσιμου αρχείου
 - είτε σε συμβολική μορφή κειμένου (*assembly code*) που μετατρέπεται σε αντικειμενική μορφή από έναν συμβολομεταφραστή (*assembler*)





Κώδικας για πραγματικές ή εικονικές μηχανές (2/4)

- Στην περίπτωση κώδικα για εικονικές μηχανές (**byte code**) ο τελικός κώδικας είναι εκτελέσιμος από την εικονική μηχανή
 - Εάν παράγετε κώδικα για υπάρχουσα εικονική μηχανή, τότε αρκεί να υπακούσετε στις προδιαγραφές του τελικού κώδικα (π.χ. Java byte code)
 - Εάν παράγετε κώδικα για εικονική μηχανή που σχεδιάσατε «εξ αρχής», πρέπει να την κατασκευάσετε εξ ολοκλήρου
 - ◆ είναι πιθανό να επηρεαστεί η παραγωγή του ενδιαμέσου κώδικα από το αρχικά προβλεπόμενο ρεπερτόριο εντολών της εικονικής μηχανής – συνήθως κάνουμε τον ενδιάμεσο κώδικα να είναι πολύ «κοντά» στην εικονική μηχανή



Κώδικας για πραγματικές ή εικονικές μηχανές (3/4)

- Η εικονική μηχανή υλοποιείται συνήθως σε μία υψηλού επιπέδου γλώσσα προγραμματισμού, με σκοπό να πετυχαίνει τη μεγαλύτερη δυνατή ταχύτητα εκτέλεσης
- Στην εργασία θα αντιμετωπίσουμε το πρόβλημα υλοποίησης μίας γλώσσας προγραμματισμού με παραγωγή κώδικα για εικονική μηχανή
 - η οποία πρόκειται θα υλοποιηθεί ως μέρος της εργασίας
 - άρα ο τελικός κώδικας θα εκτελείται μόνο από αυτή τη μηχανή
- Επιπλέον, η βιβλιοθήκη της γλώσσας θα είναι υλοποιημένη ως βασικό τμήμα της εικονικής μηχανής
 - ενώ θα είναι και επεκτάσιμη μέσω ενός ειδικού API που θα προσφέρει η υλοποίηση της εικονικής μηχανής



Κώδικας για πραγματικές ή εικονικές μηχανές (4/4)

Τα διάφορα τμήματα που σχετίζονται με την εργασία του μαθήματος για τη γλώσσα *alpha*.

<http://www.ics.forth.gr/hci/files/plang/Delta/Delta.html>

πηγαίος κώδικας
(κείμενο) στη γλώσσα
alpha



ac
alpha
Compiler (in C/C++)

τελικός κώδικας
εικονικής μηχανής
alpha



alpha
Virtual machine (in C/C++)

avm

alpha VM library extension API

alib

alpha
runtime library (in C/C++)



Περιεχόμενα

- Ο ρόλος του μεταγλωττιστή και οι φάσεις μετάφρασης
- Γλώσσες προγραμματισμού
- Τύποι και μεταβλητές
- Static and dynamic typing
- Κύκλοι επεξεργασίας στη μετάφραση
- Διερμηνείς και μεταφραστές
- Παραγωγή κώδικα για πραγματικές ή εικονικές μηχανές
- *Οι μεταγλωττιστές υπάρχουν παντού*



Οι μεταφραστές υπάρχουν παντού (1/4)

- Η χρήση μεταφραστών έχει εξαπλωθεί σε ένα πολύ μεγάλο αριθμό από πεδία εφαρμογής, γεγονός που καθιστά τη γνώση κατασκευής σε πολύτιμο «όπλο»
- Όπου εμφανίζεται η ανάγκη για περιγραφή
 - βημάτων, εντολών
 - διαδικασιών, εργασιών
 - παραμέτρων
 - οργανωμένων δομών
 - εξαρτήσεων, σχέσεων
 - δεδομένων,
 - κλπ,
- ➔ *μπορεί να σχεδιαστεί και υλοποιηθεί μία γλώσσα (δηλ. ο μεταφραστής) ώστε να περιγράφονται όλα σε κείμενο, για να μπορούν να αποθηκεύονται και τροποποιούνται πολύ εύκολα*



Οι μεταφραστές υπάρχουν παντού (2/4)

- Παραδείγματα όπου υπάρχουν ενσωματωμένοι μεταφραστές
 - Φόρμουλες σε spreadsheets (π.χ. excel)
 - Για όλων των ειδών τα configuration files
 - Query languages (SQL)
 - Shell scripts
 - Για όλες τις γλώσσες κατασκευής ιστοσελίδων και εφαρμογών για το web (HTML, UIML, JavaScript, ASP, JSP, XML, DHTML, κλπ)
 - Για όλων των ειδών τα makefiles
 - Για περιγραφή γραφικών σκηνών (VRML), true type fonts (TTFs)
 - Για περιγραφή μορφοποιημένων κειμένων (PostScript, PDF)
 - Για πρωτόκολλα επικοινωνίας σε δίκτυα

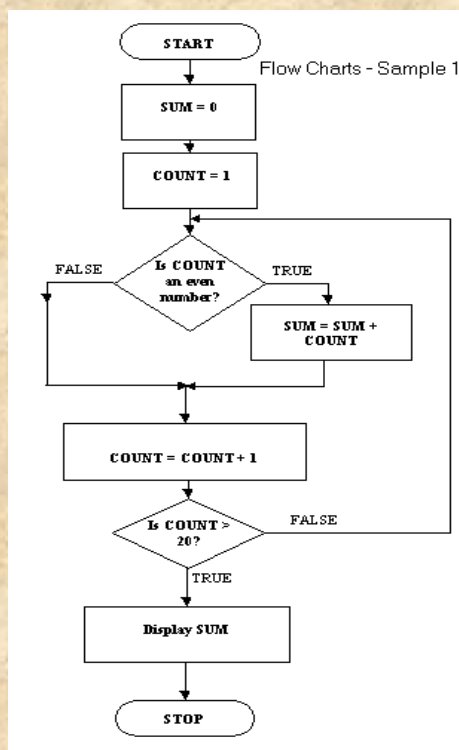


Οι μεταφραστές υπάρχουν παντού (3/4)

- Ακόμη και σε εφαρμογές που δεν το περιμένατε. Πολύ δημοφιλείς και αρκετά προηγμένες είναι οι scripting languages για την δημιουργία και επέκταση των video games
 - Unreal Script
 - ◆ Για τη σειρά τρισδιάστατων παιχνιδιών δράσης Unreal (όπως Tournament και Death Match)
 - ◆ <http://unreal.epicgames.com/UnrealScript.htm>
 - ◆ Υβριδική με χαρακτηριστικά OOP, garbage collection, static typing (δηλώνεται ο τύπος των μεταβλητών), και ειδικού σκοπού (έχει λέξεις-κλειδιά και τύπους ειδικά για games)
 - ◆ Βασίζεται σε εικονική μηχανή, δηλ. γίνεται compiled σε byte code (όμως δεν είναι «ανοικτές» οι προδιαγραφές του).
 - Quake C
 - ◆ Για τη δημιουργία των παιχνιδιών της κατηγορίας Quake (I,II,III,IV)
 - ◆ Υποτίθεται ότι μοιάζει στη C. Γίνεται compiled σε byte code ο οποίος εκτελείται από το runtime Quake engine.

Οι μεταφραστές υπάρχουν παντού (4/4)

- Τόσο οι γλώσσες όσο και οι μεταφραστές μπορεί να μην είναι «ορατοί» στους τελικούς χρήστες, αλλά να είναι ουσιαστικά ο ακρογωνιαίος λίθος ενός συστήματος. Π.χ. αυτό συμβαίνει σε συστήματα προγραμματισμού με σύμβολα – visual programming.



Διάγραμμα ροής (χρήστης)

```

terminal: start
process: sum=0;
process: count=1;
decision(1): even(count)?
true: {
    process: sum = sum-count
    goto: forward
}
false: {
    goto: forward
}
process: count = count+1;
decision(2): count=20?
true: {
    goto: forward
}
false:{
    goto: decision(1)
}
process: Display(sum)
terminal: stop
    
```

Πηγαίος κώδικας (εσωτερικά)

Τελικός κώδικας

```

sum=0;
count=1;
LABEL1:
if (even(count))
    sum = sum-count;
count = count + 1;
LABEL2:
if (!(count==20))
    goto LABEL1;
Display(sum);
    
```